

## COMPLETE PROJECT TASK BREAKDOWN

### Faster R-CNN OBB Aircraft Detection Project

I'll structure this like an actual ML project workflow so you know:

- what to do
- in what order
- what to learn
- what output each stage should give

This is the roadmap from:

**"I know nothing"**

to

**"I trained an OBB detector"**

---

## PHASE 0 — PROJECT UNDERSTANDING

### Goal

Understand:

- what you are building
  - what the final output should look like
- 

## TASK 0.1 — Understand the Problem Statement

Learn:

- What is object detection?
  - Difference between classification and detection
  - Why aircraft detection is hard
  - Why aerial imagery is difficult
  - Why rotated objects matter
- 

## TASK 0.2 — Understand Final Output

Your model should finally do:

Input Image

↓

Detect aircraft

↓

Draw rotated bounding box



Predict class/confidence

---

## **OUTPUT OF THIS PHASE**

You should be able to explain:

- What Faster R-CNN does
  - What OBB means
  - Why OBB is needed
- 

## **PHASE 1 — LEARN FOUNDATIONS**

This is the MOST IMPORTANT phase.

Do NOT skip this.

---

### **TASK 1.1 — Learn Python for Deep Learning**

Learn:

- numpy
- arrays
- tensors
- loops
- functions
- classes

Especially:

- matrix operations
- 

### **TASK 1.2 — Learn Basic Deep Learning**

Learn:

- neural networks
- forward propagation
- backpropagation
- loss functions
- gradient descent

Understand:

- how models learn
- 

### **TASK 1.3 — Learn CNNs Properly**

VERY IMPORTANT.

Learn:

- convolution
- filters
- feature maps
- pooling
- ReLU
- stride
- padding

Understand:

### **HOW CNN EXTRACTS FEATURES**

---

### **TASK 1.4 — Learn PyTorch Basics**

Learn:

- tensors
  - Dataset class
  - DataLoader
  - training loop
  - optimizer
  - loss
  - GPU training
- 

### **OUTPUT OF PHASE 1**

You should be able to:

- write simple CNN
- train on images
- understand training flow

---

## PHASE 2 — OBJECT DETECTION THEORY

Now move to detection.

---

### TASK 2.1 — Learn What Object Detection Is

Learn:

- localization
  - classification
  - detection pipeline
- 

### TASK 2.2 — Learn Bounding Boxes

Understand:

- coordinates
  - box formats
  - box regression
- 

### TASK 2.3 — Learn IoU

VERY IMPORTANT.

Understand overlap calculation.

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union}$$

---

### TASK 2.4 — Learn Metrics

Learn:

- precision
  - recall
  - mAP
  - confidence threshold
- 

### TASK 2.5 — Learn Detection Architectures

Understand:

- R-CNN
- Fast R-CNN
- Faster R-CNN

Learn:

WHY Faster R-CNN improved earlier models.

---

## **OUTPUT OF PHASE 2**

You should understand:

- how detection works
  - how models predict boxes
  - how accuracy is measured
- 

## **PHASE 3 — FASTER R-CNN DEEP UNDERSTANDING**

Now study your actual model.

---

### **TASK 3.1 — Learn Faster R-CNN Architecture**

Understand:

Image

↓

Backbone CNN

↓

Feature Map

↓

RPN

↓

ROI Pooling

↓

Classifier + Box Regressor

---

### **TASK 3.2 — Learn Region Proposal Network (RPN)**

Learn:

- anchors
  - proposal generation
  - positive/negative anchors
-

### **TASK 3.3 — Learn ROI Pooling**

Understand:

- fixed-size feature extraction
- 

### **TASK 3.4 — Learn Loss Functions**

Understand:

- classification loss
  - localization loss
- 

### **OUTPUT OF PHASE 3**

You should be able to explain:

- how Faster R-CNN detects objects
  - what RPN does
  - how boxes are refined
- 

### **PHASE 4 — OBB (ORIENTED BOUNDING BOXES)**

This is your project's special part.

---

#### **TASK 4.1 — Learn Why Normal Boxes Fail**

Aircraft are rotated.

Normal boxes:

- waste space
  - overlap incorrectly
  - reduce accuracy
- 

#### **TASK 4.2 — Learn OBB Formats**

Learn:

- 5-parameter representation

(cx, cy, w, h, angle)

- 8-point representation
-

### **TASK 4.3 — Learn Rotated IoU**

Normal IoU  $\neq$  rotated IoU.

Understand:

- angle overlap
- 

### **OUTPUT OF PHASE 4**

You should understand:

- rotated detection
  - OBB geometry
  - why OBB matters in aerial imagery
- 

### **PHASE 5 — FRAMEWORK + TOOLS SETUP**

Now practical work starts.

---

#### **TASK 5.1 — Setup Environment**

Install:

- Python
  - PyTorch
  - CUDA
  - MMCV
  - MMDetection
  - MMRotate
- 

#### **TASK 5.2 — Understand Project Structure**

Learn:

- config files
  - dataset folders
  - checkpoints
  - logs
- 

#### **TASK 5.3 — Test GPU**

Verify:

- CUDA works
  - PyTorch detects GPU
- 

## **OUTPUT OF PHASE 5**

Working environment ready.

---

## **PHASE 6 — DATASET WORK**

MOST IMPORTANT practical phase.

---

### **TASK 6.1 — Choose Dataset**

Recommended:

#### **DOTA Dataset**

Because:

- aerial imagery
  - OBB annotations
  - aircraft classes
- 

### **TASK 6.2 — Download Dataset**

Understand:

- train/val/test split
- 

### **TASK 6.3 — Understand Annotation Format**

Learn:

- label files
  - coordinates
  - class names
- 

### **TASK 6.4 — Visualize Dataset**

VERY IMPORTANT.

Draw:



- rotated boxes
- labels

This helps understand annotations deeply.

---

### **TASK 6.5 — Data Preprocessing**

Learn:

- resize
- normalization
- augmentation

Especially:

- rotation augmentation
- 

### **OUTPUT OF PHASE 6**

You should:

- fully understand dataset structure
  - visualize annotations correctly
- 

### **PHASE 7 — RUN PRETRAINED MODEL**

DO THIS BEFORE TRAINING.

---

#### **TASK 7.1 — Load Pretrained Faster R-CNN OBB**

Use MMRotate pretrained checkpoint.

---

#### **TASK 7.2 — Run Inference**

Test on:

- sample aircraft images
- 

#### **TASK 7.3 — Visualize Predictions**

Check:

- rotated boxes
- confidence scores

---

## OUTPUT OF PHASE 7

You now have:

**working detection pipeline**

Huge milestone.

---

## PHASE 8 — TRAIN YOUR MODEL

Now actual training.

---

### TASK 8.1 — Configure Training

Edit:

- classes
  - dataset path
  - batch size
  - epochs
- 

### TASK 8.2 — Start Training

Monitor:

- losses
  - GPU usage
  - learning curves
- 

### TASK 8.3 — Save Checkpoints

Understand:

- best model
  - latest model
- 

### TASK 8.4 — Debug Training Issues

Possible problems:

- exploding loss
- overfitting

- wrong annotations
- 

## OUTPUT OF PHASE 8

You now have:

**trained aircraft detector**

---

## PHASE 9 — EVALUATION

Now check quality.

---

### TASK 9.1 — Compute Metrics

Learn:

- mAP
  - precision
  - recall
- 

### TASK 9.2 — Analyze Errors

Check:

- missed aircraft
  - wrong angles
  - false positives
- 

### TASK 9.3 — Improve Model

Possible improvements:

- augmentation
  - better backbone
  - more epochs
- 

## OUTPUT OF PHASE 9

Understand:

- model strengths/weaknesses
-

## PHASE 10 — FINAL RESULTS

Project completion phase.

---

### TASK 10.1 — Generate Final Outputs

Save:

- predicted images
  - rotated detections
- 

### TASK 10.2 — Make Architecture Diagram

Include:

- Faster R-CNN flow
  - OBB explanation
- 

### TASK 10.3 — Write README

Include:

- project overview
  - setup
  - training
  - results
- 

### TASK 10.4 — Prepare Presentation

Explain:

- why OBB
  - why Faster R-CNN
  - challenges
  - results
- 

## FINAL OUTPUT OF ENTIRE PROJECT

You should finally have:

- ✓ Working Faster R-CNN OBB model
- ✓ Aircraft detection results

- ✓ Training pipeline
  - ✓ Evaluation metrics
  - ✓ GitHub project
  - ✓ Resume-worthy project
  - ✓ Understanding of modern object detection workflow
- 

## **MOST IMPORTANT LEARNING PRIORITY**

If short on time:

| <b>Priority</b>               | <b>Learn Properly</b> |
|-------------------------------|-----------------------|
| Highest                       | CNN                   |
| Highest                       | Faster R-CNN pipeline |
| Highest                       | OBB concept           |
| High                          | Dataset handling      |
| High                          | Training pipeline     |
| Medium                        | Loss math             |
| Medium                        | Anchor theory         |
| Low initially Research papers |                       |

---

## **BEST STRATEGY FOR YOU**

Do NOT:

“learn everything first”

Instead:

Learn small concept

↓

Apply immediately

↓

Continue learning

That is the fastest way to actually finish this project.